



Tarea 3: Métodos de RL para Control (2026-S1)

Problema 1: Avance del Proyecto (1 punto)

Corresponde a la *introducción del informe*. Esta sección debe incluir la justificación del problema (¿por qué es relevante?), un breve estado del arte (con al menos 4 citas bien utilizadas), y la descripción general del enfoque propuesto (técnicas que se planea utilizar).

Este avance puede ser incluido directamente en el informe de la tarea correspondiente o enviado como un documento aparte, utilizando desde ya el formato del informe final.

Problema 2: Producción de Vino Tinto (2 puntos)

Después de haber completado exitosamente la Unidad 2 de Aprendizaje Reforzado para Control de Sistemas Dinámicos, usted ha sido contratado por la *Viña Don Epsilon-Greedy* para optimizar su producción de vinos tintos.

La producción de vino tinto conlleva un proceso de fermentación en el que el azúcar presente en la uva es convertido en etanol por acción de levaduras. Este es un proceso sumamente delicado que depende principalmente de:

- **La temperatura:** A mayor temperatura aumenta la acción de las levaduras, pero estas mueren por encima de 38°C. Además, la misma reacción va produciendo calor, por lo que es importante controlar la temperatura en distintas etapas.
- **El Nitrógeno Asimilable:** Es necesario para el crecimiento de las levaduras. Sin embargo, añadir demasiado produce compuestos indeseables que pueden afectar la calidad del vino.

Por otro lado, a diferencia de los vinos blancos, los tintos fermentan en presencia de los sólidos de la uva, tales como pieles y semillas. Esto genera una transferencia de compuestos desde la fase sólida hacia la líquida, incluyendo taninos, antocianinas y otros elementos que influyen directamente en el color, aroma y calidad del vino producido. Sin embargo, producir un *buen vino* no es tan simple como maximizar una variable de proceso, la calidad del vino es evaluada por catadores, quienes consideran aspectos altamente subjetivos, complicando el control.

Para ayudar a la *Viña Don Epsilon-Greedy* a automatizar sus procesos, y ojalá superar las recetas tradicionales, se le ha dado acceso a un fermentador de prueba implementado en el script `fermentation_env.py`. El estado del sistema está dado por las siguientes variables:

$$\mathbf{x} = [S, E, X, X_d, N, \text{CO}_2, T, t] \quad (1)$$

correspondientes al azúcar residual, etanol, biomasa viable y muerta, nitrógeno asimilable, CO₂ disuelto, temperatura del mosto y tiempo transcurrido en la fermentación (en horas), respectivamente. Además, usted contará con los siguientes scripts de ayuda:

- En `expert_controller.py` encontrará un controlador experto, el cual contiene las reglas heurísticas que típicamente aplican los operarios de la viña en función de las señales medidas durante el proceso de fermentación. El controlador manipula el setpoint de temperatura y el nitrógeno asimilable.
- En `quality_functions.py` encontrará la función `maceration_extraction()`, la cual permite estimar los compuestos presentes en el vino al finalizar el proceso de fermentación. Esta función recibe como entrada una trayectoria completa de variables de estado.

- Dado que evaluar la calidad de un vino es un proceso altamente subjetivo, la viña se ha encargado de recopilar comparaciones realizadas por múltiples catadores expertos, quienes evaluaron vinos producidos de a pares. Estas comparaciones se encuentran en `data/comparisons.pkl` y consisten en una lista de diccionarios que contienen los compuestos de ambos vinos y el vino preferido por el catador. En el script `analyze_dataset.py` encontrará un ejemplo de cómo interactuar con estos datos.
- Finalmente, en `try_expert_controller.py` podrá encontrar un ejemplo de cómo instanciar el proceso, utilizar el controlador experto y extraer los compuestos resultantes.

Su Tarea

- (2 décimas)** Grafique las variables de proceso y las entradas del controlador experto para, al menos, dos condiciones iniciales distintas y calcule la calidad del vino producido utilizando la función `get_wine_quality_score`. Luego, a partir del análisis del comportamiento del controlador experto, implemente una heurística propia que sea distinta a la entregada. Grafique nuevamente la evolución del proceso, las entradas aplicadas y la calidad del vino producido utilizando `get_wine_quality_score`. Comente qué comportamientos de las entradas y qué valores de los compuestos finales parecieran estar asociados a vinos de mayor calidad (**Ojo:** `get_wine_quality_score` solo puede utilizarse para testing y evaluación final; no debe utilizarse durante el entrenamiento de sus controladores).
- (6 décimas)** Como primer paso hacia la automatización del proceso, deberá completar el script `dagger.py` implementando el algoritmo DAGGER con el objetivo de imitar el comportamiento del controlador experto. Puede inicializar su política utilizando las trayectorias disponibles en `data/expert_trajectories.py`. Una vez finalizado el entrenamiento (por una cantidad de iteraciones a su elección), deberá:
 - Comparar gráfica y numéricamente el comportamiento del controlador aprendiz y el controlador experto utilizando distintas condiciones iniciales.
 - Comparar la calidad del vino producido por ambas políticas utilizando `get_wine_quality_score`.
- (6 décimas)** Como segundo paso, queremos implementar una escala global que permita estimar qué constituye un *buen vino* únicamente a partir de las preferencias de los catadores. Para esto, deberá completar el script `fit_reward.py` y entrenar la red entregada para predecir la calidad del vino utilizando únicamente los compuestos obtenidos al terminar la fermentación. Usted deberá:
 - Dividir los datos en conjuntos de entrenamiento, validación y testeo.
 - Entrenar la red. Como **Hint**, puede basarse en el ejemplo mostrado en el código 1.
 - Sobre el conjunto de testeo, obtener la matriz de confusión (ganador predicho vs verdadero ganador) y comparar las predicciones de su red con la salida de `get_wine_quality_score`. Además, calcule el R^2 entre ambas métricas.
- (6 décimas)** Finalmente, inicializando desde la política obtenida mediante DAGGER y utilizando su red de reward, entrene un algoritmo de RL actor-critic (puede basarse en el script TD3 entregado con la próxima pregunta) para maximizar la calidad del vino producido. Considere que la red de reward solo debiese entregar una recompensa distinta de cero una vez finalizado el proceso de fermentación y calculados los compuestos finales del vino. Muestre y discuta:
 - El retorno promedio obtenido por época de entrenamiento.
 - Los scores finales predichos obtenidos por la política entrenada, las acciones y el comportamiento de los estados para algunos ejemplos.
 - Una comparación entre la política entrenada mediante RL y la política obtenida mediante DAGGER.
 - Las complicaciones asociadas a recibir una recompensa únicamente al final del episodio (*sparse rewards*) e investigue qué técnicas podrían utilizarse para mitigar este problema.

```

1 self.criterion = nn.BCEWithLogitsLoss()
2
3 ...
4 # Calculo de la perdida
5 diff = self.reward_net.get_difference_logit(A_batch, B_batch)
6 loss = self.criterion(diff, labels_batch) # labels=1 si gana A y 0 si gana B
7
8 ...
9 # Para predecir quien gana
10 logit = reward_net.get_difference_logit(A, B)
11 prob = torch.sigmoid(logit).cpu().numpy().reshape(-1) # P(A preferred)
12 pred = (prob >= 0.5).astype(float) # predicted winner
13
14 # Para predecir el score absoluto, no necesariamente en la misma escala # que el real, pero
    debería producir el mismo orden
15 score = reward_net(compounds)

```

Código 1: Entrenamiento de la red de Reward para DPO

Problema 3: Control de Drones (3 Puntos)

Como su nuevo hobby, Alan Brito ha comenzado a participar en carreras de drones autónomos, y recientemente se ha inscrito en la prestigiosa *Bellman Drone Cup*, una competencia donde únicamente se permiten drones controlados mediante algoritmos inspirados en el principio de optimalidad de Bellman. En la competencia se presentan distintas trayectorias complejas y una función de costo. El dron ganador es el que sigue la trayectoria con el menor valor de la función de costo posible.

Dentro de la comunidad existen dos corrientes de pensamiento: La primera, liderada por el reconocido Dr. Lya-punov Ramirez, cree firmemente que los modelos matemáticos y la teoría clásica de control vencerán a cualquier tipo de controlador. La segunda corriente liderada por la Dra. Bellmanova, plantea que los algoritmos de control puramente basado en datos son imbatibles.

Con el objetivo de resolver definitivamente esta disputa, Alan Brito lo ha contratado para diseñar el mejor controlador posible bajo las restricciones oficiales de la competencia. Para ello, usted contará con:

- Un modelo imperfecto de la dinámica del dron.
- Acceso limitado al dron real, pudiendo interactuar con él **a lo más en 10 intentos**.

Además, la organización de la competencia le ha entregado los siguientes scripts de apoyo:

- `Quadrotor3D.py`: Implementa las ecuaciones dinámicas del dron utilizadas tanto en simulación como en el entorno real.
- `QuadrotorTrajectories.py`: Contiene las trayectorias de referencia oficiales de la competencia.
- `LQR.py`: Implementa un controlador LQR basado en un modelo lineal imperfecto del dron, el cual puede ser utilizado para controlar el sistema real.
- `TD3.py`: Implementa una estrategia de *Domain Randomization* utilizando TD3, donde una política es entrenada sobre una distribución de parámetros simulados y posteriormente evaluada en el dron real.
- `reps_algorithm.py`: Implementa la lógica necesaria para actualizar iterativamente una distribución de parámetros, utilizando muestras previas y una medida de similitud entre las trayectorias generadas por los simuladores y las observadas en el dron real.

Su Tarea

- a) (5 décimas) Sin modificar los scripts entregados (salvo cambios menores asociados a la visualización de resultados) compare gráfica y numéricamente el desempeño del controlador LQR y del controlador TD3-DR sobre la trayectoria de referencia entregada. Para la comparación, utilice la función `reward_function_simple` como métrica de evaluación. Además, para el caso del controlador TD3:
- Muestre la evolución del retorno acumulado durante el entrenamiento.
 - Discuta por qué el control del dron resulta particularmente difícil para TD3 en este problema.
- b) (5 décimas) Dado que tiene evidencia de que tanto el approach de Lyapunov Ramirez y el de la Dra. Bellmanova tienen limitaciones, implemente un controlador residual que combine ambas técnicas de control. Continúe utilizando *Domain Randomization*.
- c) (10 décimas) Ahora, implemente *Domain Adaptation* a partir del controlador residual anterior, utilizando el código en `reps_algorithm.py`, y compare el controlador residual entrenado solo con *Domain Randomization* contra el controlador residual entrenado con *Domain Adaptation* sobre el dron real, considerando:
- Valor obtenido en la función de costo.
 - Error de seguimiento de la trayectoria.
 - Comparación de ambas trayectorias mediante gráficos 2D y 3D.
- d) (10 décimas) Finalmente, utilice la función `reward_function_complex`, disponible en `LQR.py`, e implemente una estrategia de *Domain Adaptation* en la cual la salida de la política de RL corresponda a ajustes sobre los pesos del controlador LQR alrededor de la sintonización nominal definida en `LQR.py`. Compare el LQR nominal contra el LQR sintonizado mediante TD3, considerando:
- Valor obtenido en la función de costo.
 - Error de seguimiento de la trayectoria.
 - Comparación de ambas trayectorias mediante gráficos 2D y 3D.

Anexo

Informe

El informe debe ser realizado en Latex, utilizando el template que está en Canvas.

Código

- Todo su código debe ser realizado en python version ≥ 3.11 .
- Está estrictamente prohibido el uso de Chatbots para completar su tarea. Si el profesor o los ayudantes determinan que utilizó un chatbot durante el desarrollo la tarea se calificará con Nota 1 (Ver diapositivas de la primera clase para el detalle de los lineamientos del curso en este tema).